

SDL2 in C++

The Complete Course

From Zero to Shipping Your First Game

Windows · Rendering · Input · Audio · Physics · Networking · OpenGL · Performance

Part I: Foundations	Part II: Core Systems	Part III: Game Features
Part IV: Advanced SDL2	Part V: Architecture	Part VI: Shipping & Beyond

SDL2 Tutorial Series | C++ Edition | 2024

Table of Contents

Part I — Foundations

Ch 1. Introduction to SDL2

What SDL2 is, history, ecosystem, why C++

Ch 2. Environment Setup

Installation on Win/Mac/Linux, CMake, IDE integration

Ch 3. Your First Window

Window, renderer, event loop, error handling

Ch 4. Drawing Primitives

Lines, rects, circles, polygons, blend modes

Ch 5. RAII Wrappers

unique_ptr deleters, SDL2pp, writing your own wrappers

Part II — Core Systems

Ch 6. Images and Textures

SDL_image, surface/texture pipeline, atlases

Ch 7. Input System

Keyboard, mouse, gamepad, touch, custom input map

Ch 8. The Game Loop

Delta time, fixed step, frame cap, profiling

Ch 9. Audio System

SDL_mixer deep dive, streaming, 3D audio, effects

Ch 10. Text and Fonts

SDL_ttf, Unicode, glyph caching, rich text

Part III — Game Features

Ch 11. Sprite Animation

Sprite sheets, state machines, tweening

Ch 12. Tilemaps and Levels

Tiles, layers, Tiled editor integration, parallax

Ch 13. Camera System

Viewport, zoom, shake, letterboxing, minimap

Ch 14. Collision Detection

AABB, SAT, circles, tilemaps, broadphase, response

Ch 15. Physics Basics

Gravity, friction, impulse, verlet integration

Ch 16. Particles

Emitters, pooling, GPU-side particles, effects library

Part IV — Advanced SDL2

Ch 17. SDL2 and OpenGL

OpenGL context, shaders, VBO/VAO, framebuffers

Ch 18. SDL2 and Vulkan

Surface creation, swapchain, render loop overview

Ch 19. Networking

SDL_net, TCP/UDP, lockstep multiplayer basics

Ch 20. Threading

SDL_Thread, mutexes, lock-free queues, async loading

Ch 21. Custom Renderer

Batching, draw calls, render layers, post-process

Part V — Architecture & Patterns

Ch 22. Scene Manager

Stack-based FSM, transitions, scene data

Ch 23. Entity Component System

DOD principles, components, systems, queries

Ch 24. Resource Manager

Asset pipeline, hot-reload, memory budgets

Ch 25. Event Bus

Decoupled messaging, typed events, priorities

Ch 26. UI Framework

Panels, buttons, sliders, focus, controller nav

Part VI — Shipping and Beyond

Ch 27. Debug Tools

Overlay, profiler, console, logging, asserts

Ch 28. Performance

CPU/GPU profiling, batching, texture atlases, LOD

Ch 29. Packaging

Windows installer, macOS bundle, Linux Appliance, Steam

Ch 30. Emscripten / Web

WASM build, async filesystem, canvas integration

PART I

Foundations

Core concepts, setup, and first programs

Chapter 1

Introduction to SDL2

History, ecosystem, and why it's the right choice

1.1 What is SDL2?

SDL2 (Simple DirectMedia Layer 2) is a cross-platform, MIT/zlib-licensed C library that exposes a clean, thin abstraction over platform-specific multimedia APIs. It handles windows, 2D GPU rendering, audio playback, keyboard/mouse/gamepad input, threading primitives, and high-resolution timers — all through a single portable API.

SDL2 was created by Sam Lantinga while at Loki Entertainment (Linux game ports) and later at Valve. It has been the backbone of thousands of commercial titles shipped through Steam, including Valve's own games, and is used by Humble Bundle for browser-based ports via Emscripten.

1.2 The SDL2 Ecosystem

Library	Purpose	Install Package
SDL2	Core: window, renderer, input, timer	libsdl2-dev
SDL2_image	Load PNG, JPG, BMP, TGA, WebP, AVIF	libsdl2-image-dev
SDL2_mixer	Audio: music streaming, SFX mixing	libsdl2-mixer-dev
SDL2_ttf	TrueType font rendering via FreeType	libsdl2-ttf-dev
SDL2_net	Cross-platform TCP/UDP networking	libsdl2-net-dev
SDL2_gfx	Extra primitives: filled circles, etc.	libsdl2-gfx-dev
SDL2pp	Modern C++ RAII wrapper (header-only)	via vcpkg/conan

1.3 SDL2 Architecture

SDL2 uses a driver model. Each platform provides backend drivers for video, audio, and input. Your code calls the platform-neutral SDL API; SDL dispatches to the right driver at runtime. On Windows the video driver is Direct3D 11 (or OpenGL). On macOS it is Metal. On Linux it is OpenGL via X11 or Wayland. You never need to care about the backend unless you are writing one.

```
// SDL2 rendering backend selection at runtime
// Prefer hardware; fall back to software automatically
SDL_Renderer* r = SDL_CreateRenderer(win, -1,
    SDL_RENDERER_ACCELERATED |
    SDL_RENDERER_PRESENTVSYNC);
```

```
// Query which backend was chosen
SDL_RendererInfo info;
SDL_GetRendererInfo(r, &info);
SDL_Log("Renderer: %s", info.name); // e.g. "direct3d11"
```

1.4 SDL2 vs Alternatives

Library	Language	Level	Best For
SDL2	C/C++	Low	Games needing full control
SFML	C++	Medium	C++ beginners, 2D games
Raylib	C	Low	Simple/educational projects
OpenGL/Vulkan	C/C++	Very Low	Maximum GPU control
Unity	C#	High	Commercial 2D/3D games
Godot	GDScript/C#	High	Open-source game engine

Note: SDL2 sits at the sweet spot: enough abstraction to be productive, thin enough to give you full control.

Chapter 2

Environment Setup

Installing SDL2 and configuring your build system

2.1 Windows with vcpkg + Visual Studio

```
# Step 1: Install vcpkg
git clone https://github.com/microsoft/vcpkg
cd vcpkg
bootstrap-vcpkg.bat
vcpkg integrate install

# Step 2: Install SDL2 libraries (x64-windows)
vcpkg install sdl2:x64-windows
vcpkg install sdl2-image:x64-windows
vcpkg install sdl2-mixer[mpg123,libvorbis]:x64-windows
vcpkg install sdl2-ttf:x64-windows
vcpkg install sdl2-net:x64-windows
```

2.2 Linux (Debian/Ubuntu)

```
sudo apt-get update && sudo apt-get install -y \
  libsdl2-dev libsdl2-image-dev \
  libsdl2-mixer-dev libsdl2-ttf-dev \
  libsdl2-net-dev libsdl2-gfx-dev \
  cmake ninja-build build-essential
```

2.3 macOS with Homebrew

```
brew install sdl2 sdl2_image sdl2_mixer sdl2_ttf sdl2_net cmake ninja
```

2.4 CMakeLists.txt — Full Template

```
cmake_minimum_required(VERSION 3.20)
project(MyGame LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# Find all SDL2 components
find_package(SDL2 REQUIRED)
find_package(SDL2_image REQUIRED)
find_package(SDL2_mixer REQUIRED)
find_package(SDL2_ttf REQUIRED)
find_package(SDL2_net REQUIRED)

# Glob sources (or list explicitly for larger projects)
```

```

file(GLOB_RECURSE SOURCES src/*.cpp src/*.hpp)

add_executable(${PROJECT_NAME} ${SOURCES})

target_include_directories(${PROJECT_NAME} PRIVATE src)

target_link_libraries(${PROJECT_NAME} PRIVATE
    SDL2::SDL2main
    SDL2::SDL2
    SDL2_image::SDL2_image
    SDL2_mixer::SDL2_mixer
    SDL2_ttf::SDL2_ttf
    SDL2_net::SDL2_net
)

# Copy assets to build directory
add_custom_command(TARGET ${PROJECT_NAME} POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_directory
        ${CMAKE_SOURCE_DIR}/assets ${CMAKE_TARGET_FILE_DIR:${PROJECT_NAME}}/assets)

```

2.5 VSCode launch.json Snippet

```

{
  "version": "0.2.0",
  "configurations": [{
    "name": "Debug SDL2",
    "type": "cppdbg",
    "request": "launch",
    "program": "${workspaceFolder}/build/MyGame",
    "cwd": "${workspaceFolder}/build",
    "stopAtEntry": false,
    "MIMode": "gdb"
  }]
}

```

Tip: Use `-DCMAKE_EXPORT_COMPILE_COMMANDS=ON` with CMake for clangd/intellisense support in VSCode.

Chapter 3

Your First Window

Window creation, renderer, event loop, and proper cleanup

3.1 SDL2 Initialization Flags

Flag	Subsystem Initialized
SDL_INIT_VIDEO	Window management + 2D renderer
SDL_INIT_AUDIO	Audio device + mixing
SDL_INIT_EVENTS	Event queue (auto with VIDEO)
SDL_INIT_JOYSTICK	Raw joystick input
SDL_INIT_GAMECONTROLLER	High-level gamepad API
SDL_INIT_HAPTIC	Force feedback / rumble
SDL_INIT_SENSOR	Accelerometer/gyro (mobile)
SDL_INIT_TIMER	High-resolution performance counter
SDL_INIT EVERYTHING	All of the above

3.2 Minimal Correct Program

```
#include <SDL2/SDL.h>
#include <stdexcept>
#include <string>

// Helper: throw on SDL error
static void SDLCheck(int rc, const std::string& msg) {
    if (rc != 0) throw std::runtime_error(msg + ": " + SDL_GetError());
}

static void* SDLCheckPtr(void* p, const std::string& msg) {
    if (!p) throw std::runtime_error(msg + ": " + SDL_GetError());
    return p;
}

int main(int argc, char* argv[]) {
    SDLCheck(SDL_Init(SDL_INIT_VIDEO | SDL_INIT_AUDIO), "SDL_Init");

    SDL_Window* win = (SDL_Window*)SDLCheckPtr(
        SDL_CreateWindow("SDL2 Window",
            SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED,
            1280, 720,
```

```

        SDL_WINDOW_SHOWN | SDL_WINDOW_RESIZABLE),
        "SDL_CreateWindow");

SDL_Renderer* ren = (SDL_Renderer*)SDLCheckPtr(
    SDL_CreateRenderer(win, -1,
        SDL_RENDERER_ACCELERATED | SDL_RENDERER_PRESENTVSYNC),
    "SDL_CreateRenderer");

// Logical render size (independent of window size)
SDL_RenderSetLogicalSize(ren, 1280, 720);

bool running = true;
SDL_Event ev;
while (running) {
    while (SDL_PollEvent(&ev)) {
        if (ev.type == SDL_QUIT) running = false;
        if (ev.type == SDL_KEYDOWN &&
            ev.key.keysym.sym == SDLK_F11) {
            // Toggle fullscreen
            Uint32 flags = SDL_GetWindowFlags(win);
            SDL_SetWindowFullscreen(win,
                (flags & SDL_WINDOW_FULLSCREEN_DESKTOP) ?
                0 : SDL_WINDOW_FULLSCREEN_DESKTOP);
        }
    }
    SDL_SetRenderDrawColor(ren, 30, 30, 46, 255);
    SDL_RenderClear(ren);
    SDL_RenderPresent(ren);
}
SDL_DestroyRenderer(ren);
SDL_DestroyWindow(win);
SDL_Quit();
return 0;
}

```

3.3 Logical Size and High-DPI

`SDL_RenderSetLogicalSize()` decouples your game's coordinate system from the physical window size. Draw everything in e.g. 1280x720 and SDL scales and letterboxes automatically on any resolution.

```

// Physical window 3840x2160, logical game canvas 1920x1080
SDL_RenderSetLogicalSize(ren, 1920, 1080);

// Query the actual output size (for pixel-perfect screenshots)
int ow, oh;
SDL_GetRendererOutputSize(ren, &ow, &oh);

// For high-DPI awareness on macOS:
// Add SDL_WINDOW_ALLOW_HIGHDPI to window flags

```

Note: `SDL_RenderSetLogicalSize` is one of SDL2's most underused features — it makes resolution handling trivial.

Chapter 4

Drawing Primitives

Lines, rectangles, circles, polygons, and blend modes

4.1 The Draw Color and Blend Mode

```
SDL_SetRenderDrawColor(ren, r, g, b, a);
SDL_SetRenderDrawBlendMode(ren, SDL_BLENDMODE_BLEND);

// Blend mode options:
// SDL_BLENDMODE_NONE  - overwrite destination pixel
// SDL_BLENDMODE_BLEND - standard alpha compositing
// SDL_BLENDMODE_ADD   - additive (good for glow/fire)
// SDL_BLENDMODE_MOD    - color modulation
```

4.2 Lines and Points

```
// Single line
SDL_RenderDrawLine(ren, x1, y1, x2, y2);

// Anti-aliased line (SDL2_gfx)
#include <SDL2/SDL2_gfxPrimitives.h>
aalineRGBA(ren, x1, y1, x2, y2, r, g, b, 255);

// Batch lines
SDL_Point pts[] = {{0,0},{100,200},{300,50},{400,300}};
SDL_RenderDrawLines(ren, pts, 4);

// Single / batch points
SDL_RenderDrawPoint(ren, px, py);
SDL_RenderDrawPoints(ren, pts, 4);
```

4.3 Rectangles

```
SDL_Rect r = {x, y, w, h};
SDL_RenderDrawRect(ren, &r);      // outline
SDL_RenderFillRect(ren, &r);      // solid fill

// Float variant for sub-pixel accuracy
SDL_FRect fr = {100.5f, 200.5f, 64.0f, 64.0f};
SDL_RenderDrawRectF(ren, &fr);
SDL_RenderFillRectF(ren, &fr);

// Fill whole screen (null = entire renderer)
SDL_RenderFillRect(ren, nullptr);
```

4.4 Circles and Ellipses (SDL2_gfx)

```
// Filled circle
filledCircleRGBA(ren, cx, cy, radius, r, g, b, a);

// Anti-aliased outline
aaCircleRGBA(ren, cx, cy, radius, r, g, b, a);

// Ellipse
aaEllipseRGBA(ren, cx, cy, rx, ry, r, g, b, a);

// Filled polygon
Sint16 vx[] = {100, 200, 150};
Sint16 vy[] = {100, 100, 200};
filledPolygonRGBA(ren, vx, vy, 3, r, g, b, a);
```

4.5 Custom Filled Circle (No SDL2_gfx)

```
void FillCircle(SDL_Renderer* ren, int cx, int cy, int r) {
    for (int dy = -r; dy <= r; dy++) {
        int dx = (int)sqrt((double)(r*r - dy*dy));
        SDL_RenderDrawLine(ren, cx-dx, cy+dy, cx+dx, cy+dy);
    }
}
```

4.6 Drawing a Grid

```
void DrawGrid(SDL_Renderer* ren, int cellW, int cellH,
              int screenW, int screenH) {
    SDL_SetRenderDrawColor(ren, 60, 60, 80, 100);
    SDL_SetRenderDrawBlendMode(ren, SDL_BLENDMODE_BLEND);
    for (int x = 0; x < screenW; x += cellW)
        SDL_RenderDrawLine(ren, x, 0, x, screenH);
    for (int y = 0; y < screenH; y += cellH)
        SDL_RenderDrawLine(ren, 0, y, screenW, y);
}
```

Tip: Use `SDL_BLENDMODE_ADD` for particle/glow effects and `SDL_BLENDMODE_BLEND` for HUD overlays.

Chapter 5

RAII Wrappers

Safe C++ resource management for SDL2 objects

5.1 The Problem with Raw Pointers

SDL2's C API uses raw pointers and manual cleanup functions. In C++ this leads to resource leaks when exceptions are thrown or early returns are taken. The solution is RAII (Resource Acquisition Is Initialization) — wrap every SDL resource in a class whose destructor calls the cleanup function.

5.2 `unique_ptr` with Custom Deleters

```
#include <memory>
#include <SDL2/SDL.h>

// Custom deleter types
struct SDLWindowDeleter { void operator()(SDL_Window* p){ SDL_DestroyWindow(p); } };
struct SDLRendererDeleter { void operator()(SDL_Renderer* p){ SDL_DestroyRenderer(p); } };
struct SDLTextureDeleter { void operator()(SDL_Texture* p){ SDL_DestroyTexture(p); } };
struct SDLSurfaceDeleter { void operator()(SDL_Surface* p){ SDL_FreeSurface(p); } };

// Type aliases
using WindowPtr = std::unique_ptr<SDL_Window, SDLWindowDeleter>;
using RendererPtr = std::unique_ptr<SDL_Renderer, SDLRendererDeleter>;
using TexturePtr = std::unique_ptr<SDL_Texture, SDLTextureDeleter>;
using SurfacePtr = std::unique_ptr<SDL_Surface, SDLSurfaceDeleter>;

// Usage – no manual cleanup needed
WindowPtr win{SDL_CreateWindow("Safe",
    SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED,
    1280, 720, SDL_WINDOW_SHOWN)};

RendererPtr ren{SDL_CreateRenderer(win.get(), -1,
    SDL_RENDERER_ACCELERATED | SDL_RENDERER_PRESENTVSYNC)};
```

5.3 SDL Initialisation Guard

```
class SDLSystem {
public:
    explicit SDLSystem(uint32 flags = SDL_INIT_EVERYTHING) {
        if (SDL_Init(flags) != 0)
            throw std::runtime_error(SDL_GetError());
    }
    ~SDLSystem() { SDL_Quit(); }
    // Non-copyable
    SDLSystem(const SDLSystem&) = delete;
```

```
        SDLSystem& operator=(const SDLSystem&) = delete;
};

// In main:
int main() {
    SDLSystem sdl;           // init on construction
    WindowPtr win = ...;    // exception safe
    RendererPtr ren = ...;  // exception safe
    // SDL_Quit() called automatically via SDLSystem dtor
}
```

5.4 SDL2pp — Modern C++ Wrapper

```
// sudo apt-get install libsdl2pp-dev
// or: vcpkg install sdl2pp
#include <SDL2pp/SDL2pp.hh>
using namespace SDL2pp;

SDL  sdl(SDL_INIT_VIDEO | SDL_INIT_AUDIO);
Window  win("Title", SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED, 1280, 720, SDL_WINDOW_SHO
WN);
Renderer ren(win, -1, SDL_RENDERER_ACCELERATED);

// Texture loading
SDLIMAGE::Init(IMG_INIT_PNG);
Texture  tex(ren, "player.png");
ren.Copy(tex, NullOpt, Rect(100,100,64,64));
```

Note: SDL2pp throws `SDL2pp::Exception` on any SDL error, making error handling trivial with try/catch.

PART II

Core Systems

The building blocks every game needs

Chapter 6

Images and Textures

SDL_image, surfaces, textures, atlases, and render targets

6.1 SDL_image Initialization

```
#include <SDL2/SDL_image.h>

int flags = IMG_INIT_PNG | IMG_INIT_JPG | IMG_INIT_WEBP;
if ((IMG_Init(flags) & flags) != flags)
    throw std::runtime_error(IMG_GetError());

// Cleanup (call before SDL_Quit)
IMG_Quit();
```

6.2 Loading Images Efficiently

```
// Load from file
SDL_Surface* surf = IMG_Load("assets/player.png");

// Load from memory (e.g. packed assets)
SDL_RWops* rw = SDL_RWFromMem(buffer, bufferSize);
SDL_Surface* surf2 = IMG_Load_RW(rw, 1); // 1 = auto-close

// Convert surface to GPU texture
SDL_Texture* tex = SDL_CreateTextureFromSurface(ren, surf);
SDL_FreeSurface(surf);

// Check format/size
int w, h; Uint32 fmt; int access;
SDL_QueryTexture(tex, &fmt, &access, &w, &h);
```

6.3 Texture Colour and Alpha

```
// Colour multiply (tinting)
SDL_SetTextureColorMod(tex, 255, 128, 128); // red tint
SDL_SetTextureColorMod(tex, 255, 255, 255); // reset to normal

// Alpha (0=invisible, 255=opaque)
SDL_SetTextureAlphaMod(tex, 200);

// Blend mode
SDL_SetTextureBlendMode(tex, SDL_BLENDMODE_BLEND);
```

6.4 Render Targets (Off-Screen Rendering)

```
// Create a render target texture
SDL_Texture* rt = SDL_CreateTexture(ren,
    SDL_PIXELFORMAT_RGBA8888,
    SDL_TEXTUREACCESS_TARGET,
    1280, 720);

// Draw to render target
SDL_SetRenderTarget(ren, rt);
SDL_SetRenderDrawColor(ren, 0,0,0,0);
SDL_RenderClear(ren);
/* ... draw scene ... */

// Switch back to screen
SDL_SetRenderTarget(ren, nullptr);

// Composite render target with effects
SDL_SetTextureBlendMode(rt, SDL_BLENDMODE_BLEND);
SDL_SetTextureAlphaMod(rt, 200); // global transparency
SDL_RenderCopy(ren, rt, nullptr, nullptr);
```

6.5 Texture Atlas Class

```
struct AtlasRegion { int x, y, w, h; };

class TextureAtlas {
    SDL_Texture* tex;
    std::unordered_map<std::string, AtlasRegion> regions;
public:
    TextureAtlas(SDL_Renderer* r, const std::string& imagePath,
        const std::string& dataPath);

    void draw(SDL_Renderer* r, const std::string& name,
        int x, int y, double angle=0,
        SDL_RendererFlip flip=SDL_FLIP_NONE) const {
        auto& reg = regions.at(name);
        SDL_Rect src = {reg.x, reg.y, reg.w, reg.h};
        SDL_Rect dst = {x, y, reg.w, reg.h};
        SDL_RenderCopyEx(r, tex, &src, &dst, angle, nullptr, flip);
    }
};
```

Tip: Pack all your game sprites into a single texture atlas. This reduces GPU texture binds from $O(n)$ to $O(1)$ per frame.

Chapter 7

Input System

Keyboard, mouse, gamepad, touch, and custom input mapping

7.1 Input Philosophy

SDL2 provides two mental models for input: the event model (what happened?) and the state model (what is happening right now?). Use events for one-shot actions and state for continuous actions.

7.2 Full Keyboard Handling

```
// State — checked every frame
const Uint8* keys = SDL_GetKeyboardState(nullptr);
if (keys[SDL_SCANCODE_A]) moveLeft();

// Modifiers
SDL_Keymod mods = SDL_GetModState();
bool shift = (mods & KMOD_SHIFT) != 0;
bool ctrl  = (mods & KMOD_CTRL)  != 0;

// Event — one-shot
case SDL_KEYDOWN:
    if (ev.key.repeat) break; // ignore auto-repeat
    switch (ev.key.keysym.scancode) {
        case SDL_SCANCODE_F1: toggleHelp(); break;
        case SDL_SCANCODE_GRAVE: toggleConsole(); break;
    }
    break;
case SDL_TEXTINPUT:
    // UTF-8 text for chat / console input
    inputBuffer += ev.text.text;
    break;
```

7.3 Mouse Input

```
// Absolute position + button state
int mx, my;
Uint32 mbtn = SDL_GetMouseState(&mx, &my);
if (mbtn & SDL_BUTTON(SDL_BUTTON_LEFT)) { /* LMB held */ }

// Relative motion (for FPS camera)
SDL_SetRelativeMouseMode(SDL_TRUE); // grab + hide cursor
// In event loop:
case SDL_MOUSEMOTION:
    camera.yaw  += ev.motion.xrel * sensitivity;
    camera.pitch += ev.motion.yrel * sensitivity;
```

```

        break;
    case SDL_MOUSEWHEEL:
        camera.zoom -= ev.wheel.preciseY * 0.1f;
        break;

```

7.4 Gamepad Support

```

// Enumerate and open all connected controllers
std::vector<SDL_GameController*> controllers;
for (int i = 0; i < SDL_NumJoysticks(); i++)
    if (SDL_IsGameController(i))
        controllers.push_back(SDL_GameControllerOpen(i));

// Handle hot-plug
case SDL_CONTROLLERDEVICEADDED:
    controllers.push_back(SDL_GameControllerOpen(ev.cdevice.which));
    break;
case SDL_CONTROLLERDEVICEREMOVED:
    // Remove from vector by instance ID
    break;

// Analog axes with deadzone
float ApplyDeadzone(Sint16 raw, float deadzone=0.15f) {
    float v = raw / 32767.0f;
    if (std::abs(v) < deadzone) return 0.0f;
    return (v - std::copysign(deadzone,v)) / (1.0f - deadzone);
}

```

7.5 Input Action Map

```

enum class Action { MoveLeft, MoveRight, Jump, Attack, Pause };

class InputMap {
    std::unordered_map<SDL_Scancode, Action> keyMap;
    std::unordered_map<Uint8, Action> btnMap;
    std::unordered_set<Action> held, pressed, released;
public:
    void bind(SDL_Scancode sc, Action a) { keyMap[sc] = a; }
    void bind(Uint8 btn, Action a) { btnMap[btn] = a; }

    void update(const SDL_Event& ev) {
        pressed.clear(); released.clear();
        if (ev.type == SDL_KEYDOWN && !ev.key.repeat)
            if (keyMap.count(ev.key.keysym.scancode)) {
                auto a = keyMap[ev.key.keysym.scancode];
                held.insert(a); pressed.insert(a);
            }
        if (ev.type == SDL_KEYUP)
            if (keyMap.count(ev.key.keysym.scancode)) {
                auto a = keyMap[ev.key.keysym.scancode];
                held.erase(a); released.insert(a);
            }
    }
}

```

```
    }  
    bool isHeld(Action a)    const { return held.count(a) > 0; }  
    bool isPressed(Action a) const { return pressed.count(a) > 0; }  
};
```

Chapter 8

The Game Loop

Timing, fixed steps, frame cap, and profiling

8.1 SDL High-Resolution Timer

```
Uint64 freq = SDL_GetPerformanceFrequency(); // ticks per second
Uint64 now  = SDL_GetPerformanceCounter();   // current tick

// Convert to seconds
double seconds = (double)now / (double)freq;

// Delta time between two readings
Uint64 a = SDL_GetPerformanceCounter();
// ... work ...
Uint64 b = SDL_GetPerformanceCounter();
double dt = (double)(b - a) / freq;
```

8.2 Fixed Timestep with Interpolation

```
struct GameLoop {
    static constexpr double FIXED_DT  = 1.0 / 60.0;
    static constexpr double MAX_FRAME = 0.25; // prevent spiral of death

    double accumulator = 0.0;
    Uint64 lastTime    = SDL_GetPerformanceCounter();
    Uint64 freq        = SDL_GetPerformanceFrequency();

    // Call once per OS frame
    template<typename UpdateFn, typename RenderFn>
    void tick(UpdateFn update, RenderFn render) {
        Uint64 now = SDL_GetPerformanceCounter();
        double frame = (double)(now - lastTime) / freq;
        lastTime = now;
        accumulator += std::min(frame, MAX_FRAME);

        while (accumulator >= FIXED_DT) {
            update(FIXED_DT);
            accumulator -= FIXED_DT;
        }
        render(accumulator / FIXED_DT); // alpha for interpolation
    }
};
```

8.3 Frame Rate Statistics

```

class FPSCounter {
    static const int SAMPLES = 64;
    double samples[SAMPLES] = {};
    int head = 0;
    double sum = 0;
public:
    void push(double dt) {
        sum -= samples[head];
        samples[head] = dt;
        sum += dt;
        head = (head + 1) % SAMPLES;
    }
    double avgFPS() const { return SAMPLES / sum; }
    double avgMS() const { return (sum / SAMPLES) * 1000.0; }
};

```

8.4 Frame Limiter

```

class FrameLimiter {
    Uint64 targetNS; // nanoseconds per frame
    Uint64 last = 0;
public:
    explicit FrameLimiter(int fps)
        : targetNS(1'000'000'000ULL / fps) {}

    void wait() {
        Uint64 now = SDL_GetPerformanceCounter();
        Uint64 freq = SDL_GetPerformanceFrequency();
        Uint64 elapsed = (now - last) * 1'000'000'000ULL / freq;
        if (elapsed < targetNS) {
            Uint32 ms = (Uint32)((targetNS - elapsed) / 1'000'000ULL);
            if (ms > 1) SDL_Delay(ms - 1);
            // Spin for the last millisecond for accuracy
            while ((SDL_GetPerformanceCounter() - last) * 1'000'000'000ULL
                / freq < targetNS) {}
        }
        last = SDL_GetPerformanceCounter();
    }
};

```

Tip: Use `SDL_RENDERER_PRESENTVSYNC` in combination with a frame limiter to get tear-free rendering without busy-waiting.

Chapter 9

Audio System

SDL_mixer deep dive, streaming, and 3D positional audio

9.1 Initialisation and Format Support

```
Mix_OpenAudio(48000, AUDIO_F32, 2, 1024);
Mix_Init(MIX_INIT_OGG | MIX_INIT_MP3 | MIX_INIT_FLAC | MIX_INIT_OPUS);
Mix_AllocateChannels(32);

// Query what we got
int freq, channels; Uint16 fmt;
Mix_QuerySpec(&freq, &fmt, &channels);
```

9.2 Music Streaming

```
Mix_Music* bgm = Mix_LoadMUS("music/theme.ogg");

Mix_FadeInMusic(bgm, -1, 2000);    // 2s fade in, loop forever
Mix_SetMusicPosition(0.0);         // seek to 0s
Mix_VolumeMusic(MIX_MAX_VOLUME/2);

// Pause, resume, halt
Mix_PauseMusic();
Mix_ResumeMusic();
Mix_FadeOutMusic(1000);            // 1s fade out then halt

// Callback when music ends
Mix_HookMusicFinished([]{ SDL_Log("Music ended"); });
```

9.3 Sound Effect Channels

```
Mix_Chunk* sfx = Mix_LoadWAV("sfx/explosion.wav");
Mix_VolumeChunk(sfx, 90);

// Play on any free channel
int ch = Mix_PlayChannel(-1, sfx, 0);

// Play with fade-in
ch = Mix_FadeInChannel(-1, sfx, 0, 300); // 300ms fade

// Reserve channels for specific purposes
Mix_ReserveChannels(4); // channels 0-3 reserved
Mix_GroupChannel(4, 1); // group 1 = footsteps
Mix_GroupChannel(5, 1);
Mix_PlayChannel(Mix_GroupAvailable(1), footstep, 0);
```

9.4 Positional (Pseudo-3D) Audio

```
// Panning and distance
// angle: 0=front, 90=right, 180=behind, 270=left
// distance: 0=close, 255=far (attenuated)
Mix_SetPosition(ch, angle, distance);

// Calculate angle/distance from world positions
void playSpatialSFX(Mix_Chunk* sfx,
                  float listenerX, float listenerY,
                  float sourceX,   float sourceY) {
    float dx = sourceX - listenerX;
    float dy = sourceY - listenerY;
    float dist = std::sqrt(dx*dx + dy*dy);
    float angle = std::atan2(dx, -dy) * 180.f / M_PI;
    if (angle < 0) angle += 360.f;
    Uint8 vol = (Uint8)std::clamp(255.f - dist/5.f, 0.f, 255.f);
    int ch = Mix_PlayChannel(-1, sfx, 0);
    Mix_SetPosition(ch, (Sint16)angle, 255 - vol);
}
```

Chapter 10

Text and Fonts

SDL_ttf, Unicode, glyph atlases, and rich text

10.1 Font Loading and Styles

```
TTF_Init();
TTF_Font* font = TTF_OpenFont("Roboto.ttf", 20);

// Styles
TTF_SetFontStyle(font, TTF_STYLE_BOLD | TTF_STYLE_ITALIC);

// Hinting
TTF_SetFontHinting(font, TTF_HINTING_LIGHT);

// Kerning
TTF_SetFontKerning(font, 1); // enable

// Line-height
int lineH = TTF_FontLineSkip(font);
```

10.2 Measuring Text Before Rendering

```
int w, h;
TTF_SizeUTF8(font, "Hello World", &w, &h);

// Wrap at a certain pixel width
SDL_Surface* s = TTF_RenderUTF8_Blended_Wrapped(
    font, "Long text...", {255,255,255,255}, 400);
```

10.3 Glyph Cache

```
class FontCache {
    struct Entry { TexturePtr tex; int w, h; };
    SDL_Renderer* ren;
    TTF_Font* font;
    std::unordered_map<std::string, Entry> cache;
public:
    FontCache(SDL_Renderer* r, TTF_Font* f) : ren(r), font(f) {}

    void draw(const std::string& text, int x, int y,
              SDL_Color col = {255,255,255,255}) {
        auto it = cache.find(text);
        if (it == cache.end()) {
            SDL_Surface* s = TTF_RenderUTF8_Blended(font, text.c_str(), col);
            Entry e;
```

```
        e.tex = TexturePtr{SDL_CreateTextureFromSurface(ren,s)};
        SDL_QueryTexture(e.tex.get(),nullptr,nullptr,&e.w,&e.h);
        SDL_FreeSurface(s);
        it = cache.emplace(text, std::move(e)).first;
    }
    auto& e = it->second;
    SDL_Rect dst = {x, y, e.w, e.h};
    SDL_RenderCopy(ren, e.tex.get(), nullptr, &dst);
}
void clear() { cache.clear(); }
};
```


PART III

Game Features

Animation, tilemaps, cameras, collision, physics, and particles

Chapter 11

Sprite Animation

State machines, tweening, and skeletal animation concepts

11.1 Animation Data

```
struct Frame { SDL_Rect src; float duration; };
struct Clip { std::string name; std::vector<Frame> frames; bool loop=true; };

class Animator {
    const Clip* current = nullptr;
    int frameIdx = 0;
    float timer = 0;
    bool finished = false;
public:
    void play(const Clip& clip) {
        if (current == &clip) return;
        current = &clip; frameIdx = 0; timer = 0; finished = false;
    }
    void update(float dt) {
        if (!current || finished) return;
        timer += dt;
        if (timer >= current->frames[frameIdx].duration) {
            timer -= current->frames[frameIdx].duration;
            frameIdx++;
            if (frameIdx >= (int)current->frames.size()) {
                if (current->loop) frameIdx = 0;
                else { frameIdx--; finished = true; }
            }
        }
    }
    SDL_Rect getFrame() const {
        if (!current) return {};
        return current->frames[frameIdx].src;
    }
    bool isDone() const { return finished; }
};
```

11.2 Finite State Machine Animator

```
struct AnimState {
    std::string name;
    const Clip* clip;
    std::function<std::string(const Player&)> transition;
};

class AnimFSM {
```

```

        std::unordered_map<std::string, AnimState> states;
        std::string current;
        Animator anim;
    public:
        void addState(const std::string& n, const Clip& c,
                     std::function<std::string(const Player&)> t) {
            states[n] = {n, &c, t};
            if (states.size()==1) { current=n; anim.play(c); }
        }
        void update(float dt, const Player& p) {
            auto& next = states[current].transition(p);
            if (next != current && states.count(next)) {
                current = next;
                anim.play(*states[current].clip);
            }
            anim.update(dt);
        }
        SDL_Rect frame() const { return anim.getFrame(); }
};

```

11.3 Tweening

```

// Easing functions
float easeInQuad (float t){ return t*t; }
float easeOutQuad(float t){ return t*(2-t); }
float easeInOut  (float t){ return t<.5f ? 2*t*t : -1+(4-2*t)*t; }
float easeElastic(float t){
    if(t==0||t==1) return t;
    return -pow(2,10*t-10)*sin((t*10-10.75f)*(2*M_PI)/3);
}

// Tween class
class Tween {
    float from,to,duration,elapsed=0;
    std::function<float(float)> ease;
    float* target;
public:
    Tween(float* t,float f,float to,float dur,
         std::function<float(float)> e=easeInOut)
        :target(t),from(f),to(to),duration(dur),ease(e){}
    bool update(float dt){
        elapsed+=dt;
        float t=std::clamp(elapsed/duration,0.f,1.f);
        *target=from+(to-from)*ease(t);
        return elapsed>=duration;
    }
};

```

Chapter 12

Tilemaps and Levels

Tile worlds, Tiled editor, parallax, and streaming

12.1 Tilemap Data

```
struct Tile { int id; bool solid; bool oneway; };

class Tilemap {
public:
    int width, height, tileW, tileH;
    std::vector<std::vector<int>> layers; // multiple layers
    SDL_Texture* tileset;
    int tilesetCols;

    int getTile(int layer, int tx, int ty) const {
        if (tx<0||ty<0||tx>=width||ty>=height) return -1;
        return layers[layer][ty*width+tx];
    }

    bool isSolid(int tx, int ty) const {
        int id = getTile(0, tx, ty);
        return id > 0; // non-zero = solid
    }
};
```

12.2 Tiled (.tmx) Loader (pugixml)

```
// Simple CSV layer parser
void loadTMX(Tilemap& map, const std::string& path) {
    // Parse XML with pugixml or tinyxml2
    // <layer name="Ground"><data encoding="csv">
    // 1,2,3,1,0,0,...
    // </data></layer>
    std::vector<int> layer;
    std::istringstream ss(csvData);
    std::string token;
    while(std::getline(ss, token, ','))
        layer.push_back(std::stoi(token) - 1); // Tiled IDs are 1-based
    map.layers.push_back(layer);
}
```

12.3 Parallax Background

```
struct ParallaxLayer {
    SDL_Texture* tex;
```

```
float speedX, speedY; // 0=fixed, 1=same as camera
float offsetX = 0;
};

void renderParallax(SDL_Renderer* r, ParallaxLayer& pl,
                   float camX, float camY,
                   int screenW, int screenH) {
    int tw, th;
    SDL_QueryTexture(pl.tex, nullptr, nullptr, &tw, &th);
    float ox = fmod(camX * pl.speedX, (float)tw);
    // Tile horizontally
    for (int x = -(int)ox; x < screenW; x += tw) {
        SDL_Rect dst = {x, 0, tw, th};
        SDL_RenderCopy(r, pl.tex, nullptr, &dst);
    }
}
```

Chapter 13

Camera System

Viewport, zoom, shake, letterbox, and minimap

13.1 Camera Transform

```
struct Camera {
    float x=0, y=0;      // world position of top-left
    float zoom=1;
    float rotation=0;    // degrees
    int   vpW, vpH;      // viewport (screen) size

    // World -> screen
    SDL_FPoint worldToScreen(float wx, float wy) const {
        return {(wx - x) * zoom, (wy - y) * zoom};
    }
    // Screen -> world
    SDL_FPoint screenToWorld(float sx, float sy) const {
        return {sx / zoom + x, sy / zoom + y};
    }
    // SDL_FRect in screen space
    SDL_FRect toScreenRect(float wx, float wy, float ww, float wh) const {
        auto [sx, sy] = worldToScreen(wx, wy);
        return {sx, sy, ww*zoom, wh*zoom};
    }
};
```

13.2 Camera Shake

```
class CameraShake {
    float trauma = 0; // 0-1 input, squared = shake
    float traumaDecay=2;
    float maxOffset = 20, maxAngle = 5;
    unsigned seed = 42;
public:
    void addTrauma(float t) { trauma=std::min(1.f, trauma+t); }
    void update(float dt, Camera& cam) {
        trauma = std::max(0.f, trauma - traumaDecay*dt);
        float shake = trauma*trauma;
        float n1 = ((float)rand_r(&seed)/RAND_MAX)*2-1;
        float n2 = ((float)rand_r(&seed)/RAND_MAX)*2-1;
        float n3 = ((float)rand_r(&seed)/RAND_MAX)*2-1;
        cam.x      += n1 * maxOffset * shake;
        cam.y      += n2 * maxOffset * shake;
        cam.rotation = n3 * maxAngle * shake;
    }
};
```

Chapter 14

Collision Detection

AABB, SAT, circles, tilemaps, broadphase, and response

14.1 AABB and Push-Out

```
struct AABB { float x,y,w,h; };

bool overlaps(const AABB& a, const AABB& b) {
    return a.x<b.x+b.w && a.x+a.w>b.x &&
           a.y<b.y+b.h && a.y+a.h>b.y;
}

struct MTV { float dx, dy; }; // minimum translation vector

bool resolvePenetration(AABB& a, const AABB& b, MTV& mtv) {
    float ox = std::min(a.x+a.w,b.x+b.w)-std::max(a.x,b.x);
    float oy = std::min(a.y+a.h,b.y+b.h)-std::max(a.y,b.y);
    if(ox<=0||oy<=0) return false;
    if(ox<oy){ mtv.dx=(a.x<b.x?-ox:ox); mtv.dy=0; }
    else      { mtv.dx=0; mtv.dy=(a.y<b.y?-oy:oy); }
    a.x+=mtv.dx; a.y+=mtv.dy;
    return true;
}
```

14.2 Separating Axis Theorem (Convex Polygons)

```
struct Poly { std::vector<glm::vec2> verts; };

bool SATOverlap(const Poly& a, const Poly& b) {
    auto testAxes = [&](const Poly& p) {
        for(size_t i=0;i<p.verts.size();i++){
            auto edge = p.verts[(i+1)%p.verts.size()] - p.verts[i];
            glm::vec2 axis = glm::normalize(glm::vec2(-edge.y,edge.x));
            float minA=FLT_MAX,maxA=-FLT_MAX;
            float minB=FLT_MAX,maxB=-FLT_MAX;
            for(auto& v:a.verts){float d=glm::dot(v,axis);minA=min(minA,d);maxA=max(maxA,d);}
            for(auto& v:b.verts){float d=glm::dot(v,axis);minB=min(minB,d);maxB=max(maxB,d);}
            if(maxA<minB||maxB<minA) return false;
        }
        return true;
    };
    return testAxes(a) && testAxes(b);
}
```

14.3 Tilemap Collision

```
void resolvePlayerTile(Player& p, const Tilemap& map) {
    int tileW = map.tileW, tileH = map.tileH;
    // Check 4 corners of player AABB
    int x0 = (int)(p.x) / tileW;
    int y0 = (int)(p.y) / tileH;
    int x1 = (int)(p.x + p.w - 1) / tileW;
    int y1 = (int)(p.y + p.h - 1) / tileH;
    for (int ty=y0; ty<=y1; ty++)
        for (int tx=x0; tx<=x1; tx++)
            if (map.isSolid(tx,ty)) {
                AABB tile = {(float)tx*tileW,(float)ty*tileH,
                             (float)tileW,(float)tileH};
                AABB player = {p.x,p.y,p.w,p.h};
                MTV mtv;
                if(resolvePenetration(player,tile,mtv)){
                    p.x=player.x; p.y=player.y;
                    if(mtv.dy<0) { p.vy=0; p.onGround=true; }
                    if(mtv.dy>0) p.vy=0;
                    if(mtv.dx!=0) p.vx=0;
                }
            }
}
```

Chapter 15

Physics Basics

Gravity, friction, impulse, and verlet integration

15.1 Simple Platformer Physics

```
struct RigidBody {
    float x,y, vx=0,vy=0, ax=0,ay=0;
    float mass=1, restitution=0.3f, friction=0.8f;
    bool onGround=false;
};

void integrateVerlet(RigidBody& b, float dt) {
    // Velocity Verlet
    float newX = b.x + b.vx*dt + 0.5f*b.ax*dt*dt;
    float newY = b.y + b.vy*dt + 0.5f*b.ay*dt*dt;
    // (ax,ay updated by collision/forces after this)
    b.vx += b.ax*dt; b.vy += b.ay*dt;
    b.x = newX;      b.y = newY;
}

void applyGravity(RigidBody& b, float g=980.f) {
    if (!b.onGround) b.ay = g; // pixels/s^2
}

void applyFriction(RigidBody& b) {
    if (b.onGround) b.vx *= b.friction;
}
```

15.2 Impulse Resolution

```
void resolveCollision(RigidBody& a, RigidBody& b,
                    float nx, float ny) {
    float rvx = b.vx - a.vx;
    float rvy = b.vy - a.vy;
    float velAlongNormal = rvx*nx + rvy*ny;
    if (velAlongNormal > 0) return; // separating
    float e = std::min(a.restitution, b.restitution);
    float j = -(1+e) * velAlongNormal;
    j /= 1.f/a.mass + 1.f/b.mass;
    a.vx -= j/a.mass * nx; a.vy -= j/a.mass * ny;
    b.vx += j/b.mass * nx; b.vy += j/b.mass * ny;
}
```

Chapter 16

Particle System

Emitters, pooling, and GPU-friendly design

16.1 Particle Pool

```
struct Particle {
    float x,y,vx,vy,ax,ay;
    float life, maxLife;
    Uint8 r,g,b,a;
    float size, sizeDecay;
    bool active = false;
};

class ParticlePool {
    static const int MAX = 4096;
    Particle pool[MAX];
    int head = 0;
public:
    Particle* alloc() {
        for (int i=0; i<MAX; i++) {
            int idx = (head+i) % MAX;
            if (!pool[idx].active) {
                head = (idx+1)%MAX;
                pool[idx] = {};
                pool[idx].active = true;
                return &pool[idx];
            }
        }
        return nullptr; // pool exhausted
    }

    void update(float dt) {
        for (auto& p : pool) {
            if (!p.active) continue;
            p.life -= dt;
            if (p.life <= 0) { p.active=false; continue; }
            p.vx += p.ax*dt; p.vy += p.ay*dt;
            p.x += p.vx*dt; p.y += p.vy*dt;
            float t = 1.f - p.life/p.maxLife;
            p.a = (Uint8)(255*(1-t)); // fade out
            p.size = std::max(0.f, p.size - p.sizeDecay*dt);
        }
    }

    void render(SDL_Renderer* r) {
        SDL_SetRenderDrawBlendMode(r, SDL_BLENDMODE_ADD);
        for (auto& p : pool) {
            if (!p.active || p.a==0) continue;

```

```
        SDL_SetRenderDrawColor(r,p.r,p.g,p.b,p.a);  
        SDL_FRect rc={p.x-p.size/2,p.y-p.size/2,p.size,p.size};  
        SDL_RenderFillRectF(r,&rc);  
    }  
}  
};
```


PART IV

Advanced SDL2

OpenGL, Vulkan, networking, threading, and custom rendering

Chapter 17

SDL2 and OpenGL

Setting up an OpenGL context with SDL2

17.1 Creating an OpenGL Context

```
// Request OpenGL 4.5 Core Profile
SDL_GL_SetAttribute(SDL_GL_CONTEXT_MAJOR_VERSION, 4);
SDL_GL_SetAttribute(SDL_GL_CONTEXT_MINOR_VERSION, 5);
SDL_GL_SetAttribute(SDL_GL_CONTEXT_PROFILE_MASK,
                    SDL_GL_CONTEXT_PROFILE_CORE);
SDL_GL_SetAttribute(SDL_GL_DOUBLEBUFFER, 1);
SDL_GL_SetAttribute(SDL_GL_DEPTH_SIZE, 24);
SDL_GL_SetAttribute(SDL_GL_STENCIL_SIZE, 8);
SDL_GL_SetAttribute(SDL_GL_MULTISAMPLEBUFFERS, 1);
SDL_GL_SetAttribute(SDL_GL_MULTISAMPLES, 4); // 4xMSAA

SDL_Window* win = SDL_CreateWindow("OpenGL",
                                   SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED,
                                   1280, 720,
                                   SDL_WINDOW_OPENGL | SDL_WINDOW_SHOWN);

SDL_GLContext ctx = SDL_GL_CreateContext(win);
SDL_GL_SetSwapInterval(1); // VSync

// Load GL functions (glad or glew)
// gladLoadGLLoader(SDL_GL_GetProcAddress);
```

17.2 Minimal OpenGL Draw Loop

```
while (running) {
    // Events
    while (SDL_PollEvent(&ev)) {
        if (ev.type == SDL_QUIT) running = false;
    }

    // OpenGL rendering
    glClearColor(0.1f, 0.1f, 0.18f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    /* ... draw calls ... */

    SDL_GL_SwapWindow(win); // present
}

SDL_GL_DeleteContext(ctx);
```

17.3 Shader Compilation Helper

```
GLuint compileShader(GLenum type, const char* src) {
    GLuint s = glCreateShader(type);
    glShaderSource(s, 1, &src, nullptr);
    glCompileShader(s);
    GLint ok; glGetShaderiv(s, GL_COMPILE_STATUS, &ok);
    if (!ok) {
        char log[512]; glGetShaderInfoLog(s, 512, nullptr, log);
        throw std::runtime_error(log);
    }
    return s;
}

GLuint linkProgram(GLuint vert, GLuint frag) {
    GLuint p = glCreateProgram();
    glAttachShader(p, vert); glAttachShader(p, frag);
    glLinkProgram(p);
    GLint ok; glGetProgramiv(p, GL_LINK_STATUS, &ok);
    if (!ok) {
        char log[512]; glGetProgramInfoLog(p, 512, nullptr, log);
        throw std::runtime_error(log);
    }
    return p;
}
```

Chapter 18

SDL2 and Vulkan

Surface creation and swapchain overview

18.1 Vulkan Surface via SDL2

```
SDL_Window* win = SDL_CreateWindow("Vulkan",
    SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED,
    1280, 720,
    SDL_WINDOW_VULKAN | SDL_WINDOW_SHOWN);

// Query required extensions
uint32_t extCount = 0;
SDL_Vulkan_GetInstanceExtensions(win, &extCount, nullptr);
std::vector<const char*> exts(extCount);
SDL_Vulkan_GetInstanceExtensions(win, &extCount, exts.data());

// After creating VkInstance:
VkSurfaceKHR surface;
if (!SDL_Vulkan_CreateSurface(win, instance, &surface))
    throw std::runtime_error(SDL_GetError());

// Drawable size (for swapchain extent)
int dw, dh;
SDL_Vulkan_GetDrawableSize(win, &dw, &dh);
```

Note: SDL2 handles the platform-specific surface creation (HWND, NSWindow, xcb_window_t) so your Vulkan code is fully portable.

Chapter 19

Networking

SDL_net TCP/UDP and lockstep multiplayer basics

19.1 TCP Client/Server

```
#include <SDL2/SDL_net.h>
SDLNet_Init();

// Server
IPaddress ip; SDLNet_ResolveHost(&ip, nullptr, 7777);
TCPsocket server = SDLNet_TCP_Open(&ip);
TCPsocket client = SDLNet_TCP_Accept(server); // blocking

// Client
IPaddress serverIP; SDLNet_ResolveHost(&serverIP, "127.0.0.1", 7777);
TCPsocket sock = SDLNet_TCP_Open(&serverIP);

// Send/Receive
const char* msg = "hello";
SDLNet_TCP_Send(sock, msg, (int)strlen(msg)+1);

char buf[256];
int received = SDLNet_TCP_Recv(sock, buf, 255);
```

19.2 UDP

```
UDPsocket udp = SDLNet_UDP_Open(7778);
UDPpacket* pkt = SDLNet_AllocPacket(512);

// Fill packet
memcpy(pkt->data, &gameState, sizeof(gameState));
pkt->len = sizeof(gameState);
SDLNet_ResolveHost(&pkt->address, "192.168.1.10", 7778);
SDLNet_UDP_Send(udp, -1, pkt);

// Receive
if (SDLNet_UDP_Recv(udp, pkt) > 0)
    memcpy(&remoteState, pkt->data, sizeof(remoteState));

SDLNet_FreePacket(pkt);
```

Chapter 20

Threading

SDL_Thread, mutexes, semaphores, and async asset loading

20.1 Creating Threads

```
// Thread function signature
int SDLCALL loadAssets(void* data) {
    auto* list = static_cast<std::vector<std::string*>>(data);
    for (auto& path : *list) {
        // Load assets on background thread
        // Don't call SDL rendering functions here!
    }
    return 0;
}

std::vector<std::string> paths = {"a.png", "b.ogg"};
SDL_Thread* t = SDL_CreateThread(loadAssets, "Loader", &paths);

// Wait for completion
int status;
SDL_WaitThread(t, &status);
```

20.2 Mutex and Atomic Flags

```
SDL_mutex* mtx    = SDL_CreateMutex();
SDL_atomic_t done = {0};

// Background thread
SDL_LockMutex(mtx);
sharedData.push_back(loadedTexture);
SDL_UnlockMutex(mtx);
SDL_AtomicSet(&done, 1);

// Main thread
if (SDL_AtomicGet(&done)) {
    SDL_LockMutex(mtx);
    // Consume sharedData
    SDL_UnlockMutex(mtx);
}

// Semaphore for producer-consumer
SDL_sem* sem = SDL_CreateSemaphore(0);
SDL_SemPost(sem); // signal
SDL_SemWait(sem); // wait
```


PART V

Architecture & Patterns

Scalable structures for real games

Chapter 22

Scene Manager

Stack FSM, transitions, and scene data passing

```
class Scene {
public:
    virtual ~Scene() = default;
    virtual void onEnter() {}
    virtual void onExit() {}
    virtual void onPause() {}
    virtual void onResume() {}
    virtual void handleEvent(const SDL_Event& e) = 0;
    virtual void update(double dt) = 0;
    virtual void render(SDL_Renderer* r) = 0;
};

class SceneManager {
    std::stack<std::unique_ptr<Scene>> stack;
    std::function<void()> pendingOp;
public:
    void push(std::unique_ptr<Scene> s) {
        pendingOp = [this, s=std::move(s)]() mutable {
            if (!stack.empty()) stack.top()->onPause();
            stack.push(std::move(s));
            stack.top()->onEnter();
        };
    }
    void pop() {
        pendingOp = [this]() {
            if (!stack.empty()) {
                stack.top()->onExit(); stack.pop();
                if (!stack.empty()) stack.top()->onResume();
            }
        };
    }
    void replace(std::unique_ptr<Scene> s) { pop(); push(std::move(s)); }

    void applyPending() {
        if (pendingOp) { pendingOp(); pendingOp=nullptr; }
    }
    void handleEvent(const SDL_Event& e){ if(top()) top()->handleEvent(e); }
    void update(double dt) { applyPending(); if(top()) top()->update(dt); }
    void render(SDL_Renderer* r) { if(top()) top()->render(r); }
    Scene* top(){ return stack.empty()?nullptr:stack.top().get(); }
};
```

Chapter 23

Entity Component System

DOD, dense arrays, and system queries

23.1 Sparse-Set ECS

```
using EntityID = uint32_t;
static constexpr EntityID NULL_ENTITY = UINT32_MAX;

class EntityManager {
    std::queue<EntityID> free;
    EntityID next = 0;
public:
    EntityID create() {
        if (!free.empty()) { auto id=free.front(); free.pop(); return id; }
        return next++;
    }
    void destroy(EntityID id) { free.push(id); }
};

template<typename T>
class ComponentPool {
    std::unordered_map<EntityID, T> data;
public:
    T& add(EntityID id, T comp={}){ return data[id]=std::move(comp); }
    T* get(EntityID id){
        auto it=data.find(id); return it==data.end()?nullptr:&it->second;
    }
    void remove(EntityID id){ data.erase(id); }
    auto begin(){ return data.begin(); }
    auto end() { return data.end(); }
};

// Systems iterate over components
void movementSystem(ComponentPool<Transform>& transforms,
                    ComponentPool<Velocity>& velocities, float dt) {
    for (auto& [id, vel] : velocities)
        if (auto* tr = transforms.get(id)) {
            tr->x += vel.vx * dt;
            tr->y += vel.vy * dt;
        }
}
```

Chapter 24

Resource Manager

Caching, hot-reload, and memory budgets

```
template<typename Key, typename Resource, typename Loader>
class Cache {
    std::unordered_map<Key, std::shared_ptr<Resource>> data;
    Loader loader;
public:
    explicit Cache(Loader l) : loader(std::move(l)) {}

    std::shared_ptr<Resource> load(const Key& key) {
        auto it = data.find(key);
        if (it != data.end()) return it->second;
        auto res = std::make_shared<Resource>(loader(key));
        data[key] = res;
        return res;
    }
    void unload(const Key& key) { data.erase(key); }
    void clear() { data.clear(); }
    std::size_t size() const { return data.size(); }
};

// Usage
Cache<std::string, SDL_Texture*, TextureLoader> texCache{TextureLoader{ren}};
auto tex = texCache.load("player.png");
```

Chapter 25

Event Bus

Decoupled typed events between systems

```
class EventBus {
    using Handler = std::function<void(const void*)>;
    std::unordered_map<std::size_t, std::vector<Handler>> handlers;
public:
    template<typename E>
    void subscribe(std::function<void(const E)> fn) {
        handlers[typeid(E).hash_code()].push_back(
            [fn](const void* e){ fn(*static_cast<const E*>(e)); });
    }
    template<typename E>
    void emit(const E& event) {
        auto it = handlers.find(typeid(E).hash_code());
        if (it != handlers.end())
            for (auto& fn : it->second) fn(&event);
    }
};

// Usage
struct PlayerDied { int score; };
bus.subscribe<PlayerDied>([](const PlayerDied& e){
    showGameOver(e.score);
});
bus.emit(PlayerDied{1234});
```


PART VI

Shipping and Beyond

Debug tools, performance, and distribution

Chapter 27

Debug Tools

Overlay HUD, logging, console, and assertions

27.1 Debug Overlay

```
class DebugOverlay {
    TTF_Font* font;
    SDL_Renderer* ren;
    int y = 5;
public:
    void beginFrame() { y = 5; }
    void print(const std::string& text) {
        SDL_Color c = {200,200,50,255};
        SDL_Surface* s = TTF_RenderUTF8_Solid(font, text.c_str(), c);
        SDL_Texture* t = SDL_CreateTextureFromSurface(ren, s);
        int w,h; SDL_QueryTexture(t,nullptr,nullptr,&w,&h);
        SDL_Rect dst={5,y,w,h}; SDL_RenderCopy(ren,t,nullptr,&dst);
        SDL_FreeSurface(s); SDL_DestroyTexture(t);
        y += h + 2;
    }
};

// Usage each frame
overlay.beginFrame();
overlay.print("FPS: " + std::to_string((int)fps.avgFPS()));
overlay.print("Entities: " + std::to_string(ecs.count()));
overlay.print("Draw calls: " + std::to_string(renderer.drawCalls));
```

Chapter 28

Performance

Profiling, batching, atlases, and culling

28.1 GPU Batch Renderer

Every `SDL_RenderCopy` is a draw call. For hundreds of sprites, batch them into a single `SDL_RenderGeometry` call using SDL2's geometry API (available since SDL 2.0.18):

```
// Build vertex/index arrays
std::vector<SDL_Vertex> verts;
std::vector<int> indices;

auto addSprite = [&](SDL_FRect dst, SDL_Rect src, int tw, int th,
                  SDL_Color tint) {
    int base = (int)verts.size();
    float u0 = src.x/(float)tw, v0 = src.y/(float)th;
    float u1 = (src.x+src.w)/(float)tw;
    float v1 = (src.y+src.h)/(float)th;
    verts.push_back({{dst.x, dst.y, tint, {u0,v0}}});
    verts.push_back({{dst.x+dst.w, dst.y, tint, {u1,v0}}});
    verts.push_back({{dst.x+dst.w, dst.y+dst.h, tint, {u1,v1}}});
    verts.push_back({{dst.x, dst.y+dst.h, tint, {u0,v1}}});
    indices.insert(indices.end(), {base, base+1, base+2, base, base+2, base+3});
};

// Draw everything in one call
SDL_RenderGeometry(ren, atlasTexture,
    verts.data(), (int)verts.size(),
    indices.data(), (int)indices.size());
```

28.2 Frustum Culling

```
bool isVisible(const Camera& cam, float wx, float wy, float ww, float wh){
    float sx=wx-cam.x, sy=wy-cam.y;
    return sx+ww>0 && sx<cam.vpW/cam.zoom &&
           sy+wh>0 && sy<cam.vpH/cam.zoom;
}
```

Chapter 29

Packaging

Windows installer, macOS bundle, Linux AppImage, Steam

29.1 Windows — Collecting DLLs

```
# CMake post-build: copy all runtime DLLs automatically
# In CMakeLists.txt:
if(WIN32)
    add_custom_command(TARGET MyGame POST_BUILD
        COMMAND ${CMAKE_COMMAND} -E copy_if_different
            $<TARGET_RUNTIME_DLLS:MyGame>
            $<TARGET_FILE_DIR:MyGame>
        COMMAND_EXPAND_LISTS)
endif()
```

29.2 macOS App Bundle

```
set_target_properties(MyGame PROPERTIES
    MACOSX_BUNDLE TRUE
    MACOSX_BUNDLE_BUNDLE_NAME "My Game"
    MACOSX_BUNDLE_BUNDLE_VERSION "1.0.0"
    MACOSX_BUNDLE_GUI_IDENTIFIER "com.studio.mygame"
)

# Fix dylib paths after build:
# install_name_tool -change @rpath/SDL2.dylib
#     @executable_path/../Frameworks/SDL2.dylib MyGame
```

29.3 Linux AppImage

```
# linuxdeploy auto-bundles .so files
linuxdeploy --appdir AppDir \
    --executable MyGame \
    --plugin sdl2 \
    --output appimage
```

29.4 Emscripten / WebAssembly

```
# Configure with emcmake
emcmake cmake .. -DCMAKE_BUILD_TYPE=Release
make -j4

# Required linker flags in CMakeLists.txt
if(EMSCRIPTEN)
    target_link_options(MyGame PRIVATE
```

```
-sUSE_SDL=2 -sUSE_SDL_IMAGE=2 -sUSE_SDL_TTF=2 -sUSE_SDL_MIXER=2
-sUSE_SDL_MIXER=2 --preload-file assets
-sALLOW_MEMORY_GROWTH=1 -sINITIAL_MEMORY=67108864
-sASYNCIFY # needed for blocking SDL_Delay on web
)
endif()
```

Chapter 30

Emscripten / Web

The main loop, async filesystem, and canvas integration

30.1 Browser Main Loop

Browsers don't support a blocking while loop. Emscripten's `emscripten_set_main_loop()` calls your tick function once per rAF (requestAnimationFrame).

```
#include <emscripten.h>
#include <emscripten/html5.h>

static Game* g_game = nullptr;

void mainLoopTick() {
    g_game->tick();
}

int main() {
    static Game game;
    g_game = &game;
    game.init();
#ifdef __EMSCRIPTEN__
    emscripten_set_main_loop(mainLoopTick, 0, 1);
#else
    while (game.isRunning()) game.tick();
#endif
    return 0;
}
```

30.2 Async File Loading

```
// Preload files at HTML load time (--preload-file assets/)
// or fetch asynchronously:
emscripten_async_wget(
    "https://example.com/level2.tmx",
    "/level2.tmx",
    [](const char* file){ /* loaded */ },
    [](const char* file){ /* error */ }
);
```

Congratulations!

You have completed the full SDL2 in C++ course. You now understand every layer of a real-time game: window management, hardware-accelerated 2D rendering, full input handling, audio, text, animation, tilemaps, camera systems, collision detection, physics, particles, OpenGL integration, Vulkan setup, networking, threading, entity-component systems, and shipping to every platform. The best next step is to build a complete game — even a small, finished one teaches more than any tutorial. Good luck!

— *SDL2 Tutorial Series | C++ Edition*